Trabajo Práctico Final - Predicción de Consumo Energético Industrial

Objetivo General

Desarrollar un pipeline de machine learning completo y reproducible para predecir el consumo energético del sistema de refrigeración `Frio (kW)` del día siguiente en una planta cervecera mexicana. Este proyecto no solo evaluará la precisión del modelo, sino también la implementación de buenas prácticas de MLOps para garantizar la trazabilidad, colaboración y gestión del ciclo de vida del proyecto.

Dataset

Se proporcionan archivos Excel con registros horarios de métricas operacionales de la planta desde 2021 hasta 2024:

- `Totalizadores Planta de Cerveza 2023_2024.xlsx`
- `Totalizadores Planta de Cerveza - 2022_2023.xlsx`
- `Totalizadores Planta de Cerveza 2021_2022.xlsx`
- `Totalizadores Planta de Cerveza 2024_2025.xlsx` no proporcionado (**Test**)

Variable Objetivo (Target): Frio (kW)

El objetivo del modelo es predecir el consumo eléctrico total diario del sistema de refrigeración (Frio (kW)) para el día siguiente. Para lograrlo, se deben seguir dos pasos clave de preprocesamiento:

Paso 1: Calcular el Consumo Total Diario Es crucial entender que las variables en los archivos son **totalizadores horarios**. Esto significa que el valor de cada hora representa el consumo acumulado desde el inicio del día hasta ese momento.

Para obtener el **consumo total de cada día**, deben filtrar los datos y quedarse con el último valor registrado de la jornada. Ese valor final representa la suma de todo el consumo del día y será el que utilicen como característica (`X`) para esa fecha.

Cuidado: las 23:00 y las 23:59 no son lo mismo. Muchos días tienen registros en ambas horas, y la diferencia promedio entre los dos totalizadores es del orden de los 800 kW — alrededor del 20 % del margen de error que se les permite. Elijan una de las dos definiciones, aplíquenla de manera consistente en todo el trabajo, y justifíquenla en el informe.

Paso 2: Construir la Variable a Predecir (`y`) Una vez que tengan los datos agregados por día (el valor de las 23:59 de cada jornada), deben construir la variable objetivo. Como el objetivo es predecir el consumo del día *siguiente*, es necesario alinear las características de un día `D` con el consumo del día `D+1`.

Implementación Recomendada: Sobre el dataframe de datos diarios, utilicen la función `df.shift(-1)` de pandas en la columna `Frio (kW)` para crear su columna target `y`.

Fase 0: Configuración del Entorno y Control de Versiones

Antes de iniciar el análisis, es fundamental establecer un entorno de trabajo robusto y versionado.

0.1 Gestión de Dependencias y Entorno Reproducible

- **Crear un entorno virtual aislado** para el proyecto. Se recomienda utilizar `conda` en combinación con `uv` para una gestión de dependencias eficiente:

  Bash

  ```
  # Crear el entorno con conda
  conda create -n cervecera_env python=3.12
  ```

```
conda activate cervecera_env

# Instalar uv para gestionar paquetes
pip install uv

# Instalar librerías usando uv
uv pip install pandas numpy scikit-learn jupyter xgboost lightgbm
```

- **Generar un archivo de dependencias** `requirements.txt` que documente todas las librerías instaladas con sus versiones específicas.

  Bash

  ```
  uv pip freeze > requirements.txt
  ```

0.2 Sistema de Control de Versiones con Git

- **Inicializar un repositorio Git distribuido** en la carpeta raíz del proyecto.
- **Configurar un repositorio remoto** en una plataforma como GitHub o GitLab y sincronizar el proyecto.
- **Crear un archivo `README.md`** detallado que incluya instrucciones para clonar el repositorio, configurar el entorno virtual a partir del `requirements.txt` y ejecutar los scripts principales.

0.3 Estrategia de Branching (Git Flow) Todo el desarrollo se realizará siguiendo una estrategia de branching. La rama `main` debe permanecer siempre estable. Se utilizarán ramas `feature` para cada fase del proyecto:

- `feature/eda`
- `feature/preprocessing`
- `feature/modeling`
- `feature/prediction-pipeline`

Cada funcionalidad debe ser desarrollada en su propia rama y luego integrada a una rama `develop` (o directamente a `main` para este proyecto) mediante **Pull Requests**.

Fase 1: Análisis Exploratorio de Datos (EDA)

Trabajar sobre la rama `feature/eda`.

El conjunto de datos con el que trabajaremos contiene una **gran cantidad de variables**. El desafío consiste en realizar un análisis exploratorio inicial *inteligente* para simplificar el conjunto de datos, apoyándose tanto en métodos estadísticos como en el **conocimiento del dominio**.

Para ello, se sugiere fuertemente realizar un análisis detallado, **hoja por hoja del Excel**, formulando hipótesis sobre qué procesos (Elaboración, Envasado, Servicios Auxiliares, etc.) pueden estar más relacionados con el consumo del sistema de refrigeración. Durante esta exploración, presten especial atención a dos aspectos clave:

1. La **variabilidad** de las métricas: Aquellas variables con poca o nula variación a lo largo del tiempo difícilmente puedan explicar los cambios en el consumo y suelen aportar poca información al modelo.
2. La **correlación** con la variable objetivo: Identifiquen y cuantifiquen qué variables muestran una relación (directa o inversa) más fuerte con el consumo de `Frio (kW)`, ya que serán las candidatas principales para el modelo predictivo.

El objetivo final es justificar la selección de un subconjunto de características que sea a la vez manejable e informativo.

Realizar un análisis exhaustivo del dataset que incluya:

1.1 Análisis Descriptivo y Versionado Inicial de Datos

- Unificar todos los archivos Excel en un único DataFrame. **Atención: los archivos se solapan en el tiempo.** Cuantifiquen cuántas fechas aparecen en más de un archivo, verifiquen si el total diario coincide entre ellos, y definan y justifiquen una regla de desempate. Verifiquen además que ninguna fecha tenga más de 25 registros antes de agregar por día: si alguna los tiene, hay más de una jornada apilada bajo la misma etiqueta y el `shift(-1)` va a quedar desalineado.
- **Implementar un sistema de checksums**: Calcular y guardar el hash **SHA-256** del dataset unificado inicial, leyendo el archivo por bloques (ver la función `huella()` del capítulo 10). Almacenar el par `{archivo: sha256}` en `data/checksums.json`. Justifiquen en una línea por qué no se usa MD5.
- Dimensiones del dataset completo.
- Tipos de variables y rangos de valores.
- Identificación y cuantificación de valores faltantes.
- Detección de valores atípicos y de registros inválidos: días con menos de 24 lecturas horarias (el totalizador final de un día incompleto **no** representa el consumo de esa jornada), valores imposibles del orden de los millones, y días en que el totalizador no se reinició a medianoche.

1.2 Análisis Temporal

- Evolución del consumo de `Frio (kW)` a lo largo del tiempo.
- Identificación de patrones estacionales (mensuales y anuales).
- Análisis de patrones diarios y semanales.
- **Midan** el efecto del día de la semana sobre el consumo, en lugar de asumirlo. Si el efecto resulta despreciable, díganlo con el número en la mano y descarten esas variables justificando la decisión: descartar con evidencia es un resultado, no una omisión.

1.3 Análisis de Correlaciones

- Matriz de correlación entre variables numéricas.
- Identificación de las 15 variables más correlacionadas con el target.
- Análisis de multicolinealidad entre predictores.
- Visualización mediante heatmaps.

1.4 Visualizaciones Requeridas

- Serie temporal del consumo de `Frio (kW)`.
- Boxplots mensuales para identificar estacionalidad.
- Distribución del target (histograma y QQ-plot).
- Scatter plots de las 5 variables más correlacionadas vs. target.
- Análisis de consumo por área de la planta (Elaboración, Envasado, etc.).

Entregable: Notebook `eda.ipynb` con análisis completo y **conclusiones**. **Realizar un Pull Request** para integrar los cambios a la rama principal.

Fase 2: Preprocesamiento y Selección de Variables

Trabajar sobre la rama `feature/preprocessing`.

2.1 Limpieza de Datos

- Tratamiento de los registros inválidos detectados en la Fase 1: días incompletos, totales imposibles y días sin reinicio del totalizador.
- Manejo de valores faltantes.
- Eliminación o corrección de outliers extremos.
- **Documentar todas las decisiones** de limpieza en el notebook, justificando cada elección.

2.2 Feature Engineering

- Creación de variables derivadas sugerencias:
  - Agregaciones temporales (promedios móviles).
  - Variables de lag.
  - Día de la semana.
  - Mes del año.
  - Indicador de fin de semana (binario).
  - Ratios entre consumos de diferentes áreas.
  - Eficiencia energética (consumo real contra meta). Las metas están en las columnas que empiezan con `Meta` dentro de la hoja `Consolidado KPI`, expresadas por hectolitro.
  - Interacciones entre variables principales.

2.3 Selección de Variables

- Aplicar al menos 2 métodos de selección (RFE, Importancia de variables con Random Forest, LASSO, etc.).
- Justificar el conjunto final de variables seleccionadas.

2.4 Preparación y Versionado de Datos Procesados

- **Documentar el linaje de datos**: Crear un registro (ej. `data/processed/data_lineage.json`) que trace las transformaciones aplicadas, incluyendo el script utilizado y la fecha.
- Guardar el conjunto de datos final y limpio en `data/processed/dataset_final.csv`. Calcular y añadir su checksum al archivo `checksums.json`.
- **División temporal de datos: Train 70 % (el tramo más antiguo), Validation 15 %, Test 15 % (el más reciente).** La selección de modelo y de hiperparámetros se hace **exclusivamente** sobre Validation, o con `TimeSeriesSplit` sobre Train. El conjunto de Test se toca **una sola vez**, al final, para reportar. Elegir modelo mirando el test es la fuga de datos que estudiaron en el capítulo 4.

- Normalización/Estandarización de variables, **dentro de un `Pipeline` de scikit-learn ajustado únicamente con los datos de entrenamiento**. El archivo que guarden en `data/processed/` va codificado y depurado pero **sin escalar**, para no dejar información del test dentro de un artefacto versionado.
- Documentar el shape final de X_train, X_val, X_test.

Entregables:

- Notebook `preprocesamiento.ipynb`.
- Script `src/preprocessing_pipeline.py` con el pipeline completo y reproducible.
- **Realizar un Pull Request** para integrar los cambios.

Fase 3: Entrenamiento y Versionado de Modelos

Trabajar sobre la rama `feature/modeling`.

3.1 Modelos Requeridos **Antes que cualquier modelo, dos baselines obligatorios.** No son un trámite: son la vara contra la que se mide todo lo demás, y sin ellos un MAE es un número sin significado.

- **Persistencia**: `ŷ[D+1] = y[D]`, es decir "mañana consume lo mismo que hoy".
- **Media móvil de 7 días**: `ŷ[D+1]` = promedio de los siete días anteriores.

Repórtenlos con el mismo MAE, sobre el mismo conjunto y con el mismo preprocesamiento que su modelo final. Los dos se implementan en pocas líneas de pandas.

Luego, entrenar y comparar al menos 4 modelos:

- **XGBoost**
- **Random Forest Regressor**
- **LightGBM**
- **Ridge/Lasso Regression**

3.2 Optimización y Seguimiento de Experimentos

- **Implementar un sistema de tracking de experimentos**: Registrar automáticamente en un archivo `results/experiment_logs.csv` los hiperparámetros, métricas (MAE, RMSE, R^2), y un timestamp para cada ejecución de entrenamiento.
- Documentar el espacio de búsqueda y justificar la elección del mejor modelo basándose en el análisis de los logs de experimentos.

3.3 Evaluación de Modelos

- Comparar performance en el conjunto de validación.
- Análisis de residuos y gráficos de predicción vs. valores reales.
- Feature importance del mejor modelo.

3.4 Modelo Final y Registro de Modelos

- Seleccionar el mejor modelo basado en MAE de validación.
- Reentrenar con train+validation.
- **Implementar versionado semántico**: Guardar el modelo final serializado (ej. `joblib`) con una versión semántica clara (ej. `models/modelo_v1.0.0.pkl`).
- **Crear un model registry local**: En un archivo `models/model_registry.json`, registrar la versión del modelo, sus hiperparámetros finales, métricas de validación, la ruta al archivo `.pkl` y el hash del commit de Git con el que fue entrenado.

Entregables:

- Notebook `modelado.ipynb`.
- Script `src/train_model.py`.
- Archivo del modelo `modelo_v1.0.0.pkl` (o la versión correspondiente).
- Archivo `model_registry.json`.
- **Realizar un Pull Request** para integrar los cambios.

Fase 4: Pipeline de Predicción

Trabajar sobre la rama `feature/prediction-pipeline`.

Crear un script `src/predict.py` que automatice la predicción sobre nuevos datos.

Python

```
# Estructura requerida del script
import pandas as pd
import joblib
import sys
import json

def load_model(version='latest'):
    """Carga el modelo y los preprocesadores desde el registro.

    'latest' carga el modelo con la version mas alta.
    """
    with open('models/model_registry.json', 'r') as f:
        registry = json.load(f)

    # Logica para encontrar la ultima/mejor version
    model_path = registry[version]['path']
    model = joblib.load(model_path)
    return model

def load_and_preprocess(filepath):
    """Carga el archivo Excel y aplica todo el preprocesamiento.

    Reusa el pipeline de preprocessing_pipeline.py: no lo reimplementa.
    Devuelve la matriz de caracteristicas y las fechas correspondientes.
    """
    # ... aplicar pipeline de preprocesamiento ...
```

```
    return X_processed, fechas

def predict_consumption(filepath):
    """Funcion principal de prediccion."""
    model = load_model()  # carga desde el registry
    X, fechas = load_and_preprocess(filepath)
    predictions = model.predict(X)

    # La prediccion es DIARIA: una fila por dia, no por hora.
    # 'fecha' es el dia D; la prediccion corresponde al consumo del dia D+1.
    return pd.DataFrame({
        'fecha': fechas,
        'prediccion_frio_kw': predictions
    })

if __name__ == "__main__":
    results = predict_consumption(sys.argv[1])
    results.to_csv('results/predicciones.csv', index=False)
    print("Predicciones generadas en results/predicciones.csv")
```

Requisitos del script:

- Debe cargar la versión del modelo desde el `model_registry.json`.
- Debe aplicar exactamente el mismo pipeline de preprocesamiento que en el entrenamiento.
- Debe manejar errores y generar un archivo CSV con las predicciones.
- **Contrato con el archivo de entrada.** El archivo de test tiene la misma estructura general que los que reciben, pero **no necesariamente las mismas hojas ni las mismas columnas**: puede tener hojas de más y columnas de más. Por lo tanto el script debe acceder a hojas y columnas **por nombre, nunca por posición**, no debe asumir el orden ni la cantidad de columnas, y debe fallar con un mensaje claro —diciendo qué falta— si no encuentra alguna de las que necesita. Un script que revienta con un `KeyError` sin explicación no cumple la condición de aprobación.

Entregable: Script `src/predict.py` funcional. **Realizar un Pull Request final**.

Conclusiones

El informe cierra respondiendo estas cinco preguntas. No se responden con lo que hicieron sino con lo que **decidieron**, y cada respuesta tiene que apoyarse en un número que ustedes midieron.

1. **¿Su modelo agrega valor?** Pongan en una tabla el MAE de los dos baselines y el de su modelo final, los tres sobre el mismo conjunto de test. Si la mejora sobre el mejor baseline es menor al 10 %, digan honestamente qué están comprando con toda la complejidad que agregaron —tiempo de entrenamiento, dependencias, un pipeline que hay que mantener— y si la comprarían.
2. **¿Qué variables se parecen demasiado al objetivo?** Hay variables en el dataset que correlacionan por encima de 0,98 con `Frio (kW)` del mismo día, porque miden el mismo sistema físico desde otra hoja. Decidan si entran o no a su modelo, y en la justificación distingan dos cosas distintas: usar el valor de **hoy** para predecir mañana es legítimo; usar el de mañana no lo es. Digan cuál de los dos casos es el suyo y cómo se aseguraron.
3. **Un resultado que contradice la intuición.** El enunciado y la documentación del dataset afirman que hay un patrón semanal. Midieron el efecto en la Fase 1. Si su medición no coincide con lo que el enunciado afirma, escríbanlo con el número, expliquen qué pudo haberlo borrado, y digan qué hicieron en consecuencia.

4. **Defiendan un umbral.** ¿Qué diferencia mínima de MAE, expresada en kW, consideran que justifica reemplazar un modelo ya desplegado por otro? El número no puede salir de la nada: deriven de algún lado —el desvío entre folds, la variabilidad del target, el costo de equivocarse— y díganlo.

5. **Lo que no usaron.** Este proyecto atraviesa las cinco unidades, pero ustedes no aplicaron todo lo que vieron. Elijan **dos** técnicas de los prácticos 1 a 5 que decidieron **no** usar acá —PCA, MICE, codificación cíclica, LOF, DVC, lo que sea— y para cada una expliquen por qué no correspondía. Descartar con criterio también es una decisión de ingeniería.

Modalidad y entregas

Grupos de dos o tres integrantes. No se aceptan trabajos individuales. `git log --oneline` debe mostrar commits de todos los integrantes, distribuidos en el tiempo.

El trabajo tiene **cuatro entregas**:

	Cuándo	Qué se entrega
Entrega 1	semana 8	Presentación de ideas
Entrega 2	semana 9	Propuesta formal
Entrega 3	semana 12	Presentación y defensa del código
Entrega 4	semana 14	**Defensa final**

La **defensa final** es oral, ante la cátedra, con el proyecto corriendo. Al comenzar esa clase todo el código debe funcionar: no se corrigen errores durante la defensa. **Cada integrante debe poder explicar y justificar cualquier parte del trabajo, no solamente la que escribió.**

Se entrega además un **informe** con las decisiones tomadas, su justificación y las limitaciones del modelo.

Reproducibilidad. Declaren `SEMILLA = 42` y úsenla en toda partición y en todo estimador que acepte `random_state`. Dos ejecuciones sucesivas tienen que dar los mismos números.

Criterios de Evaluación

Condición de Aprobación

Las cuatro son eliminatorias:

1. **Reportar el MAE de los dos baselines** (persistencia y media móvil de 7 días) sobre el mismo conjunto de test que su modelo final, y **mejorarlos**. Un trabajo sin los dos baselines reportados no aprueba, tenga el MAE que tenga.
2. **MAE < 3000** en el set de test oculto.
3. El proyecto debe ser completamente reproducible: `git clone`, `uv pip install -r requirements.txt` y la ejecución de los scripts deben funcionar sin errores en una máquina que no es la de ustedes.
4. El script `predict.py` debe ejecutarse sin errores, y todas las fases deben estar reflejadas en el historial de Git con sus ramas y Pull Requests.

Distribución del puntaje

Componente	Puntos
Fase 0 — entorno, control de versiones y trazabilidad	15
Fase 1 — análisis exploratorio y conclusiones	20
Fase 2 — preprocesamiento, ingeniería y selección de variables	25
Fase 3 — modelado, baselines, experimentos y registro de modelos	25
Fase 4 — pipeline de predicción	15

Componente	Puntos
Total	**100**

Se aprueba con 60 puntos, siempre que se cumplan las cuatro condiciones eliminatorias.

Penalizaciones

- Código no reproducible o entorno mal configurado: −20.
- Falta de documentación (README, comentarios, justificaciones): −10.
- No seguir la estructura de control de versiones y branching solicitada: −10.
- Fallo en la implementación del versionado de datos, tracking de experimentos o registro de modelos: −10.
- Seleccionar el modelo mirando el conjunto de test: −15.

Estructura de Entrega Final

```
tp_final/
├── data/
│   ├── Totalizadores Planta de Cerveza*.xlsx
│   ├── checksums.json
│   └── processed/
│       ├── dataset_final.csv
│       └── data_lineage.json
├── models/
│   ├── modelo_v1.0.0.pkl
│   └── model_registry.json
├── notebooks/
│   ├── eda.ipynb
│   ├── preprocesamiento.ipynb
│   └── modelado.ipynb
├── src/
│   ├── preprocessing_pipeline.py
│   ├── train_model.py
│   ├── predict.py
│   └── auxiliar_functions.py
├── results/
│   ├── predicciones.csv
│   └── experiment_logs.csv
├── .gitignore
├── README.md
├── requirements.txt
└── consigna.md
```

Requisitos Técnicos

- **Python 3.12**, entorno aislado creado con `conda` + `uv`, y `requirements.txt` congelado con `uv pip freeze`.
- El proyecto se clona y se ejecuta siguiendo únicamente el `README.md`. Si hace falta un paso que no está escrito ahí, el proyecto no es reproducible.
- **Semilla fija** (`SEMILLA = 42`) en toda partición y en todo estimador que acepte `random_state`.
- Los cuatro archivos `.xlsx` **no van al repositorio**: se declaran en `.gitignore` y el `README.md` explica de dónde se obtienen. Lo que sí se versiona es `data/checksums.json`, que permite verificar que quien clona tiene exactamente los mismos archivos.
- El modelo serializado se versiona con nombre semántico (`models/modelo_v1.0.0.pkl`) y el commit correspondiente se etiqueta con `git tag -a v1.0.0`, de modo que la versión del registro y la del repositorio sean la misma.

Recomendaciones

1. **Validación Temporal**: Respetar el orden temporal de los datos

   - No usar datos futuros para predecir pasado
   - Considerar que el modelo se usará para predicción del día siguiente

2. **Feature Engineering**: El consumo de frío depende de:

   - Temperatura ambiente (implícita en estacionalidad). La planta esta ubicada en la ciudad de Tecate, Baja California, Mexico Se recomienda `Open-Meteo`
   - Volumen de producción
   - Etapa del proceso de fermentación
   - Día de la semana (patrones de producción)

3. **Interpretabilidad**: Documentar:

   - Por qué se eligió cada transformación
   - Importancia de variables en el modelo final
   - Limitaciones del modelo

   Nota: Este trabajo simula un caso real de la industria. La calidad del código, documentación y reproducibilidad serán evaluadas como en un entorno profesional.